Illia Kolotnitskyi

Simulation and Classsification of Driver Behaviour based on Vehicle Trajectory Data

Bachelor's Thesis to achieve the university degree of Bachelor Software Engineering and Management

submitted to

Graz University of Technology

Supervisor Roman Kern

Institute of Machine Learning and Neural Computation Head: Univ.-Prof. Dipl.-Ing. Dr.techn. Robert Legenstein

Advisor Felix Hofbaur

Graz, Juli 2025

Acknowledgments

I would like to express my sincere gratitude to Felix Hofbaur for his detailed explanations of the Intelligent Driver Model (IDM) and his valuable sessions, which greatly enhanced my understanding of the parameters and their applications.

I am also deeply thankful to Roman Kern for his guidance and supervision throughout the development of this thesis.

Abstract

Understanding and modeling driver behavior is essential for realistic traffic simulations and the development of advanced driver-assistance systems. The Intelligent Driver Model (IDM) is often used to simulate longitudinal vehicle motion, but its performance depends strongly on how accurately its parameters are calibrated to real-world data.

This thesis examines the effect of different calibration strategies on IDM parameters and explored whether the calibrated parameters could be used to classify driver behavior. First, several error functions including absolute, relative, and mixed error were applied to calibrate the IDM parameters for individual driver trajectories. Then, the calibrated parameters were analyzed using unsupervised learning methods, including rule-based thresholding and k-means clustering.

The results show that the choice of error function significantly influences the quality and distribution of the calibrated parameters. In particular, the relative error function often produced unrealistic acceleration values, while the mixed error provided more balanced outcomes. The classification experiments revealed that despite overlapping parameter distributions, certain behavioral patterns could be identified and linked to plausible driving styles.

This work demonstrates that driver classification based on IDM parameters is feasible and can provide insights into driver behavior beyond simple model fitting. However, there are limitations: the approach is not suitable for all drivers, especially those with frequent lane changes, as the IDM assumes uninterrupted car-following. Despite this, the methodology can support more realistic traffic simulations and potentially guide the design of adaptive driver-assistance systems.

Contents

1 Introduction **1**

2 Background **3**
2.1 Driver Behavior and Traffic Analysis 3
2.2 Trajectory Data 3
2.3 Car-Following Models 3
2.4 Parameter Calibration 4
2.5 Behavior Classification Scheme 4

3 Related Work **5**
3.1 Literature Review Methodology 5
3.1.1 IDM Fundamentals 5
3.1.2 Parameter Calibration 6
3.1.3 Behavior Classification 6
3.2 Research Gap 7

4 Use Cases & Requirements **8**
4.1 Use Case Overview 8
4.2 Functional Requirements 8
4.3 Non-Functional Requirements 9

5 Problem Statement **10**
5.1 Given Input 10
5.2 Formal Objective 10
5.3 Model Equations 11
5.4 Problem Summary 11

6 Methods **12**
6.1 Parameter Estimation 12
6.1.1 Parameter Bounds 12
6.1.2 Fitness Function 13
6.1.3 Error Evaluation 14
6.1.4 Genetic Algorithm 15

6.1.5 Maximum Likelihood Estimation 16
6.1.6 Differential Evolution 17
6.2 Driver Classification 18
6.2.1 Rule-Based Classification 19
6.2.2 K-Means Clustering 19
6.3 Visualization 20

7 Evaluation **22**
7.1 Overview and Baseline 22
7.2 Data Description 22
7.3 Calibration Setup 23
7.4 Evaluation Metrics 24
7.5 Best Performing Calibration Strategy 24
7.6 Results 26
7.6.1 Calibration Results 26
7.6.2 Classification Results 28
7.7 Discussion 36
7.7.1 Interpretation of Results 36
7.7.2 Classification Analysis 37
7.7.3 Limitations 40
7.7.4 Future Work 41

8 Conclusions **42**

List of Figures

7.1 Gap error across calibration strategies 25
7.2 Position error across calibration strategies 25
7.3 Acceleration error across calibration strategies 26
7.4 Driver class distribution by rule-based classification (lane-changing) 29
7.5 Driver class distribution by K-means clustering (lane-changing) . 30
7.6 3D visualization of K-means clusters in parameter space (lane-changing) 31
7.7 Pairwise distributions of IDM parameters across K-means clusters (lane-changing) 32
7.8 Driver class distribution by rule-based classification (no lane changes) 33
7.9 Driver class distribution by K-means clustering (no lane changes) 34
7.10 3D visualization of K-means clusters in parameter space (no lane changes) 35
7.11 Pairwise distributions of IDM parameters across K-means clusters (no lane changes) 36
7.12 Distribution of driving styles using rule-based classification, comparing lane-changing and non-lane-changing scenarios. 38
7.13 K-means parameters for Aggressive drivers. 38
7.14 K-means parameters for Normal drivers. 39
7.15 K-means parameters for Cautious drivers. 39

List of Algorithms

1 Genetic Algorithm (GA) Calibration 15
2 Maximum Likelihood Estimation (MLE) Calibration 16
3 Differential Evolution (DE) Calibration 17

1 Introduction

Understanding driver behavior is essential for traffic simulation, safety analysis, and the development of autonomous systems. While many car-following models exist, the Intelligent Driver Model (IDM) is among the most widely used due to its simplicity and interpretability. The accuracy of IDM-based simulations heavily depends on the proper calibration of its parameters to reflect actual human driving behavior.

Only in recent years has it become possible to study this problem with high precision. Advances in drone-based traffic observation now provide detailed, large-scale trajectory datasets that capture vehicle movements at high resolution. At the same time, IDM and its objective functions have been studied more thoroughly, enabling a better understanding of how parameter choices affect model behavior. This combination of accurate data and deeper theoretical insight makes it timely to revisit IDM calibration and explore new applications such as driver classification.

In this work, IDM parameters are calibrated for individual trajectories using different error functions. A minimum of 235 valid frames per vehicle ID, roughly ten seconds of driving data at the dataset's 25 frames per second is used to ensure sufficient information for reliable fitting. The calibrated parameters are then analyzed to identify patterns in driving behavior, and unsupervised learning methods are applied to group drivers based on these parameters. The relationships between these groups and typical driving styles are examined, providing a data-driven perspective on driver behavior.

The main research questions are how different objective functions influence the quality of IDM calibration, whether the calibrated IDM parameters can be used to classify drivers in a meaningful way, and what insights into driving behavior can be gained from the structure of the calibrated parameters.

The contributions of this thesis are as follows. First, a calibration pipeline is built using multiple error functions, including absolute, relative, and mixed error. Second, the impact of each error function on model performance is compared. Third, an unsupervised classification of drivers is carried out based on calibrated parameters, and the resulting clusters are interpreted from a behavioral perspective.

This work builds on recent research in driver behavior modeling but adds an additional layer by linking calibrated parameters to classification. Since driver behavior plays a central role in traditional traffic systems as well as in the devel-

opment of autonomous driving technology, the ability to extract behavior patterns from trajectory data is highly relevant. The findings in this thesis may help improve traffic simulations and driver-assistance systems.

2 Background

2.1 Driver Behavior and Traffic Analysis

Understanding driver behavior is fundamental for improving traffic safety, optimizing traffic flow, and developing advanced driver assistance systems and autonomous vehicles. Drivers behave differently, some change lanes often and accelerate quickly, while others keep larger distances and drive more calmly. This work focuses on analyzing such behaviors using trajectory data.

2.2 Trajectory Data

Modern traffic analysis relies on trajectory data, typically stored in comma-separated value (.CSV) files containing time-series records of vehicle positions, speeds, and other state variables. In this study, data was collected using a drone that filmed an approximately 400 meter section of highway with two lanes of traffic in each direction.

Each entry includes the vehicle's position coordinates (x, y), current speed (v), and additional metadata such as lane position and vehicle class (car or truck). The data tracks how vehicles move relative to each other over time. These positional relationships serve as direct inputs for calculating acceleration profiles through the Intelligent Driver Model (IDM).

2.3 Car-Following Models

The Intelligent Driver Model (IDM), first introduced by Treiber et al. (2000) and extensivly formalized in Treiber and Kesting (2012), represents a fundamental advancement in microscopic traffic modeling. This car-following model predicts a driver's acceleration v_{IDM} through:

$$v_{IDM} = a\left[1 - \left(\frac{v}{v_0}\right)^4 - \left(\frac{s^*(v, \Delta v)}{s}\right)^2\right] \tag{2.1}$$

where v_{IDM} is the calculated acceleration, v is the current speed, s is the distance to the leading vehicle and the desired gap s^* .

Each parameter reflects specific aspects of driving style. For example, a high desired speed (v_0) suggests a preference for fast driving, while a small time gap (T) indicates desire of the driver to follow closely. The maximum acceleration (a) and comfortable deceleration (b) further characterize how abruptly the driver changes speed. Together, these parameters create a fingerprint of individual driving behavior.

2.4 Parameter Calibration

The main challenge lies in calibrating the IDM parameters to match the real driver behavior. For each vehicle trajectory, a set of parameters is searched such that the acceleration (v_{IDM}) predicted by the model is close to the acceleration (v_{XAcc}) actually measured by the driver. This is formulated as an optimization problem that minimizes the difference between v_{IDM} and v_{XAcc} at all time frames.

The resulting parameters characterize how closely a driver follows the IDM's idealized behavior, forming the basis for subsequent classification.

2.5 Behavior Classification Scheme

The classification of driver behavior requires careful consideration of both traffic safety requirements and computational practicality. While existing literature proposes various classification systems (Bouhsissin et al., 2023), our work adopts a three-class framework (aggressive, normal, cautious) for fundamental reasons to improve and make the clustering clearer.

These behavior categories are defined by distinct car-following characteristics derived directly from IDM parameter optimization:

1. **Aggressive**: Characterized by small time gaps, high acceleration rates, and abrupt maneuvers
2. **Normal**: Follows traffic flow with moderate acceleration and safe following distances
3. **Cautious**: Maintains large gaps, accelerates gently, and avoids abrupt actions

This combination of behavioral modeling, optimization techniques, and efficient data processing enables effective classification of driver behavior patterns from real-world trajectory data, as demonstrated in our results.

3 Related Work

3.1 Literature Review Methodology

This literature review was conducted using a systematic approach to identify relevant work in the field of driver behavior classification using car-following models. The primary sources were publications provided by the Institute for Road and Transportation Engineering. These were complemented by targeted searches on Google Scholar, using key terms such as "Intelligent Driver Model", "IDM Calibration", "Parameters Calibration" and "Driver Behavior Classification".

From the total pool of identified literature, eight publications formed the basis for detailed analysis. These works were categorized into four core research phases: (1) IDM fundamentals, (2) parameter calibration and (3) behavior classification. Table 3.1 summarizes the thematic focus of each phase.

Table 3.1: Research phases and corresponding literature focus

Research Phase	**Key Aspects**
1. IDM Fundamentals	Core model formulation, parameter interpretation and typical applications in traffic simulation
2. Parameter Calibration	Methods for estimating IDM parameters from trajectory data, including error functions and validation approaches
3. Behavior Classification	Frameworks for translating IDM parameters into driver behavior categories

3.1.1 IDM Fundamentals

The basis of this work lies in the original formulation of the Intelligent Driver Model (IDM) by Treiber et al. (2000), introduced as a time-continuous, collision-free car-following model. The authors demonstrated that IDM could reproduce a

wide range of real traffic phenomena, including free flow, congestion, and stop-and-go waves, by using local variations of a single parameter to represent bottlenecks.

Empirical validation of the IDM was later conducted also by Kesting and Treiber (2008) using real trajectory data from radar-equipped vehicles in urban traffic. They applied nonlinear optimization methods to fit model parameters to observed data and found the IDM to be more robust and consistent across different calibration criteria than other models such as the velocity difference model. Furthermore, their validation across multiple trajectories showed that variability within the same driver was a more significant source of error than differences between drivers.

A full theoretical overview is provided in Treiber and Kesting (2012)'s textbook, which explains the IDM acceleration equation and the physical meaning of its five key parameters: desired speed (v_0), time gap (T), desired minimum distance (s_0), maximum acceleration (a), and comfortable deceleration (b). The book also gives typical value ranges for each parameter under different traffic scenarios and vehicle types, forming the reference for calibration and classification in this thesis.

3.1.2 Parameter Calibration

Initial efforts to calibrate IDM parameters were provided by Treiber and Kesting (2013), who developed a comprehensive framework for assessing the descriptive and predictive capabilities of car-following models. Their approach included both local maximum likelihood and global least-squares estimation (LSE) methods, applied to real and synthetic trajectory data. A key contribution of their work was the detailed analysis of various objective functions used for calibration, which directly influenced the robustness and interpretability of the estimated parameters.

This foundational work was extended by Alhariqi et al. (2022), who introduced the adaptive IDM, a model variant that allows real-time parameter adjustment based on prevailing traffic conditions. Their study focused on mixed traffic settings that included both human-driven and autonomous vehicles. Importantly, they incorporated the standard deviation of velocity into the calibration objective function to better capture stop-and-go traffic dynamics.

3.1.3 Behavior Classification

Recent work has focused on classifying driver behavior using parameters from IDM output, including desired speed (v_0), time gap (T), desired distance (s_0), and comfortable deceleration (b). Wen et al. (2022) applied PCA and hierarchical clustering on high-resolution driving data to group drivers into four behavior types:

aggressive and non-aggressive at both low and high speeds. This showed how clustering can reveal clear behavior patterns in mixed traffic with autonomous vehicles.

Bouhsissin et al. (2023) reviewed existing methods for driver classification and highlighted the use of rule-based systems built on features like speed, spacing, and acceleration. These are closely related to IDM parameters and are important for building classification scheme.

While Stoler et al. (2024) focused on scenario generation rather than classification, their results support the value of realistic, parameter-driven modeling for capturing human-like driving behavior.

3.2 Research Gap

The research covered demonstrates that IDM is a widely accepted and empirically validated model for the simulation of driver behavior. Its parameters have been used successfully in behavior classification and traffic flow analysis.

However, one major limitation remains: the IDM in its standard or adaptive form does not incorporate mechanisms for modeling lane changes. The experiments described in Alhariqi et al. (2022) were conducted on a single-lane circular road, focusing exclusively on longitudinal dynamics. While this setup avoids the complexity of lane changes, it also means the model has not been tested in multi-lane traffic.

This gap is particularly relevant for classification tasks using long trajectory segments in naturalistic traffic. In such scenarios, lane changes can cause abrupt spacing and velocity variations, which may be unrealistic by the IDM as changes in car-following style. If lane changes are not explicitly handled, the stability of the parameter estimates can be affected, which directly impacts the accuracy and reliability of driver behavior classification.

4 Use Cases & Requirements

This work involves the analysis and classification of driver behavior using calibrated IDM parameters from real vehicle trajectory data. The technical implementation consists of several Python-based components: (1) data calibration, (2) behavior classification, and (3) visualization of results. The following sections summarize the key requirements of the system.

4.1 Use Case Overview

The Python-scripts process pre-recorded trajectory data and output calibrated parameters for each driver and categorized driver behavior profiles. It follows three main steps:

- **Calibration Script:** Extract and optimize IDM parameters for each driver using data segments.
- **Classification Script:** Assign each driver to a behavior category based on calibrated parameters.
- **Visualization Script:** Plot classification results and statistical summaries for reporting and evaluation.

4.2 Functional Requirements

- **F1. Data Import:** The system must load vehicle trajectory data from ‘.pkl‘ and ‘.csv‘ files containing time-series segments.
- **F2. IDM Calibration:** The system must perform parameter estimation using the Intelligent Driver Model and store calibrated parameters for each driver.
- **F3. Behavior Classification:** The system must classify driver behavior using rule-based thresholds and k-means clustering.

- **F4. Result Storage:** Classification outputs and parameters must be saved in separate '.cvs' for further analysis.
- **F5. Visualization:** The system must generate clear plots (trajectory comparisons, acceleration comparisons, parameter distributions, cluster visualizations) in '.png' format for reporting and evaluation.

4.3 Non-Functional Requirements

- **NF1. Reproducibility:** All scripts must produce consistent results when executed with the same data and configuration.
- **NF2. Modularity:** Each script must be modular and independently executable, supporting reusability and debugging.
- **NF3. Compatibility:** The implementation must run under a standard Python environment (Python 3.11) with commonly available libraries (NumPy, pandas, scikit-learn, matplotlib).
- **NF4. Performance:** Scripts must complete processing of a standard dataset (1 Recording with approximately 150 driver ids) within reasonable time (under 10 minutes).
- **NF5. Interpretability:** Visualizations and outputs must be human-readable and support direct interpretation of driver types.

5 Problem Statement

5.1 Given Input

The input to this work consists of 60 high-resolution drone recordings included in the highD dataset, covering a 400-meter section of highway. Each recording includes a `.pkl` file with time-series vehicle trajectory data and a corresponding `.csv` file containing per-vehicle metadata. From the '.pkl' files, we extract only the necessary fields required for IDM calibration: the ego vehicle's velocity v, longitudinal acceleration $XAcc$, and the distance headway s to the leading vehicle. We also compute the relative speed Δv to the preceding vehicle, where $\Delta v = v - v_L$, with v_L being the preceding vehicle's velocity. If no preceding vehicle is detected, we assign a default gap of $s = 250$ meters and a fixed $\Delta v = 1.0$ m/s to preserve consistency.

The '.csv' files provide each vehicle's class, either "Car" or "Truck", which is used to set bounds during parameter optimization. All recordings operate at 25 frames per second (FPS).

5.2 Formal Objective

The calibration process is performed separately for each driver in the dataset. For a given driver, we extract a sequence of observed values over T time steps: ego velocity v_t, gap to the leading vehicle s_t, relative velocity Δv_t, and measured longitudinal acceleration a_t^{meas}. These values form a driver-specific trajectory dataset:

$$D_{\text{driver}} = \{(v_t, s_t, \Delta v_t, XAcc_t)\}_{t=1}^{T}$$

The goal is to find a parameter set

$$\theta = (T, a, b, v_0)$$

such that the IDM-generated accelerations a_t closely approximate the observed accelerations $XAcc_t$ and result in a realistic reproduction of the observed gaps over time.

5.3 Model Equations

The acceleration produced by the IDM is given by:

$$v_{IDM} = a\left[1 - \left(\frac{v}{v_0}\right)^4 - \left(\frac{s^*(v, \Delta v)}{s}\right)^2\right] \tag{5.1}$$

where the desired dynamic distance is:

$$s^*(v, \Delta v) = s_0 + vT + \frac{v\Delta v}{2\sqrt{ab}} \tag{5.2}$$

The model parameters include: the minimum spacing s_0 (typically set to 2 m), desired time gap T, maximum acceleration a, comfortable deceleration b, and desired speed v_0. These parameters collectively determine the driver's acceleration behavior in different traffic situations.

5.4 Problem Summary

Formally, given input trajectories D, the task is to find:

$$f : D \to \theta = (T, a, b, v_0)$$

such that the IDM-generated acceleration and position closely match the observed values, minimizing one of the above error functions.

The estimated parameters are later used for clustering and rule-based classification of drivers into *Aggressive*, *Normal*, or *Cautious* groups.

6 Methods

This section describes the methods used to estimate the four IDM parameters from recorded driving data, classify driver types, and visualize the results.

6.1 Parameter Estimation

We estimate parameters of the IDM for each vehicle using three optimization strategies: Genetic Algorithm (GA), Maximum Likelihood Estimation (MLE), and Differential Evolution (DE). All algorithms aim to minimize an objective function that quantifies the deviation between simulated and observed inter-vehicle gaps over time.

Given the recorded vehicle trajectories

$$D_{\text{driver}} = \{(v_t, s_t, \Delta v_t, XAcc_t)\}_{t=1}^{T}$$

for a single driver over T frames, the goal is to find optimal IDM parameters:

$$\theta = (T, a, b, v_0)$$

6.1.1 Parameter Bounds

Before running any optimization, we define the search space for all four IDM parameters. The safe time headway T is bounded between 0.5 and 5.0 seconds. This reflects realistic following behavior from aggressive to normal driving. Maximum acceleration a and comfortable deceleration b are both constrained within 0.1 to 5.0 m/s², covering the typical capabilities of cars and trucks under normal and extreme conditions. The desired speed v_0 is not globally fixed, but dynamically computed per driver. It ranges from the maximum speed observed for that vehicle during the recording to 5% above that value. This approach helps prevent the optimizer from underestimating a driver's actual intent, especially when when only limited trajectory data is available.

To keep these bounds realistic for each vehicle type, we also enforce global constraints: for cars, v_0 must stay within 80–150 km/h, and for trucks between 60–120 km/h. These bounds are converted to m/s internally. If the observed maximum speed violates these global bounds, it is clipped accordingly.

6.1.2 Fitness Function

To evaluate a parameter candidate θ, we simulate the follower vehicle's motion frame by frame. At each time step, the IDM acceleration is recalculated based on the current velocity, gap, and relative speed to the leader. We assume a fixed time resolution of $\Delta t = \frac{1}{25}$ s, corresponding to the dataset's 25 FPS sampling rate.

The fitness functions, originally proposed by Kesting and Treiber (2008), compare the simulated inter-vehicle gap s_{sim} to the observed gap s_{data} at each frame. From the first frame, we extract initial values for the follower and leader. The follower starts at position $x = 0$, while the leader's position is initialized at $x = s_0$, the initial observed gap. Then for each frame, we update the follower's velocity and position using the IDM formula, while the leader's velocity is updated directly from the data. The simulated inter-vehicle gap is computed as the difference between the leader's and follower's simulated positions.

For each time step, three types of error terms are computed to assess the quality of the simulated gap. The **relative error** f_{rel} quantifies how much the simulated gap deviates from the observed one in proportion to the observed value. The **absolute error** f_{abs} captures the raw difference between simulated and observed gaps, normalized by the average observed gap across the time series. Finally, the **mixed error** f_{mixed} combines absolute error with scaling based on the individual observed gap size, offering a hybrid measure that balances both local and global sensitivity.

The angle brackets $\langle \cdot \rangle$ denote the arithmetic mean over all time steps in the trajectory. The mean is used to make the error measures not be centered at some specific moment, and it is a better measure of model performance because it is more stable and representative.

These are computed as follows:

$$f_{\text{rel}} = \sqrt{\left\langle \left(\frac{s_{\text{sim}} - s_{\text{data}}}{s_{\text{data}}} \right)^2 \right\rangle}$$

$$f_{\text{abs}} = \sqrt{\frac{\langle (s_{\text{sim}} - s_{\text{data}})^2 \rangle}{\langle s_{\text{data}} \rangle^2}}$$

$$f_{\text{mixed}} = \sqrt{\frac{1}{\langle s_{\text{data}} \rangle} \cdot \left\langle \frac{(s_{\text{sim}} - s_{\text{data}})^2}{s_{\text{data}}} \right\rangle}$$

If any simulation step leads to invalid values (negative speed, NaNs), the function returns infinity to indicate failure. We also discard parameter vectors where a, b, or v_0 are non-physical (negative values).

6.1.3 Error Evaluation

To assess the quality of the estimated IDM parameters, we compute three evaluation metrics: acceleration error, position error, and gap error. These serve as post-hoc validation metrics and are not used during optimization. All errors are calculated per driver and per algorithm and then aggregated.

Gap Error

The objective function in parameter optimization minimizes the error between the simulated and observed inter-vehicle gap at each frame. For validation, the mean absolute gap error is re-calculated per driver by directly comparing the observed gap s_t with the simulated gap s_t^* over all time steps t. This is computed separately for each algorithm as:

$$E_{\text{gap}} = \frac{1}{N} \sum_{t=1}^{N} |s_t - s_t^*|$$

where N is the total number of frames for the given driver.

These values are then grouped by driver to compute mean per-driver errors. This metric directly reflects the optimization objective and ensures internal consistency between training and evaluation.

Acceleration Error

For each driver and algorithm, the observed longitudinal acceleration a_{Xacc} is compared with the model-predicted IDM acceleration a_{IDM}. The absolute difference is calculated for each frame, and the mean value over all frames is used as the error metric:

$$E_{\text{acc}} = \frac{1}{N} \sum_{t=1}^{N} |a_{\text{Xacc}}(t) - a_{\text{IDM}}(t)|$$

where N is the number of frames with valid (finite) acceleration values.

This metric helps identify parameter sets that produce physically unrealistic or unstable accelerations.

Position Error

We simulate the trajectory of the follower vehicle using IDM-generated acceleration and integrate it over time with a fixed step size ($dt = \frac{1}{25}$s). The resulting positions are compared to actual positions derived from recorded $XAcc$. Root

Mean Squared Error (RMSE) is used to quantify the deviation, and it is normalized by the average actual position:

```
rmse = np.sqrt(np.mean((sim_pos - actual_pos)**2))
error_percent = 100 * rmse / np.mean(np.abs(actual_pos))
```

Simulations that fail due to invalid values or instability are skipped. This metric complements acceleration error by validating how well the estimated model reproduces long-term vehicle behavior.

Together, these three metrics provide a comprehensive view of model accuracy across both instantaneous and accumulated driving behavior.

6.1.4 Genetic Algorithm

Algorithm 1 Genetic Algorithm (GA) Calibration

```
Initialize a population of random parameter vectors within predefined bounds
for each generation do
    Evaluate fitness of each individual
    Select candidates using tournament selection
    Create offspring through crossover
    Apply mutation with a fixed probability
end for
return best individual found across all generations
```

The Genetic Algorithm (GA) is a population-based optimization method inspired by biological evolution Holland (1992). It is well suited for noisy or non-differentiable objective functions, making it appropriate for IDM calibration.

In each generation, the fitness of every individual is computed as the deviation between simulated and observed gaps using the chosen error function.

Following Ali (2023), tournament selection is applied, where three candidates are sampled and the one with the lowest fitness is chosen, balancing exploration and exploitation. Pairs of selected individuals produce offspring through crossover, where each parameter is either inherited from one parent or averaged with equal probability.

Mutation is applied to each parameter with a fixed probability, adding Gaussian noise proportional to the parameter's range. All parameters are clipped to remain within their respective bounds. This cycle of evaluation, selection, crossover, and mutation is repeated for a fixed number of generations.

The best-performing individual found during the entire run is returned as the final calibrated parameter set.

The population initialization of the implemented GA is:

```
# population init
population = []
for _ in range(pop_size):
    population.append([
        random.uniform(*T_range),
        random.uniform(*a_range),
        random.uniform(*b_range),
        random.uniform(*v0_range)
    ])
```

GA is useful when the search space has many local minima or when gradient-based methods might fail, but it requires tuning of hyperparameters like population size, mutation rate, and number of generations.

6.1.5 Maximum Likelihood Estimation

Algorithm 2 Maximum Likelihood Estimation (MLE) Calibration

Define parameter bounds based on vehicle class and observed $v_{\max}$
for each of 3 restarts **do**
  Initialize random starting point x_0 within bounds
  Run L-BFGS-B minimization using x_0
  Update best result if current solution is better
end for
return best parameter vector found

Maximum Likelihood Estimation (MLE) uses the L-BFGS-B algorithm from the scipy.optimize package to find a local minimum of the fitness function. It is a deterministic, gradient-based optimizer that performs well on smooth functions, Madej (2021) notes that it has limitations particularly for complex distributions where some parameters must be known a priori or when alternative estimation methods may be required.

To mitigate this, we start the search from multiple random initial points within the valid bounds. Each starting point triggers a separate optimization run. The final output is the parameter vector that achieved the best (lowest) fitness value across all attempts.

Each run uses the following structure:

```
for _ in range(3):  # try 3 different starting points
```

```
x0 = [
    random.uniform(low, high)
    for (low, high) in bounds
]

 result = minimize(
    fun=fitness,
    x0=x0,
    args=(frames, s0, vehicle_class),
    bounds=bounds,
    method='L-BFGS-B',
    options={'maxiter': 100}
)
```

MLE is more efficient in smooth, well-behaved regions of the parameter space and usually converges faster than population-based methods. It remains a preferred choice when approximate parameter ranges are known and computational efficiency is prioritized over global optimality guarantees.

6.1.6 Differential Evolution

Algorithm 3 Differential Evolution (DE) Calibration

Define parameter bounds based on vehicle class and observed $v_{\max}$
Initialize population with size proportional to parameter dimensions
for each generation up to max iterations **do**
  for each individual in population **do**
    Generate mutant vector by adding scaled difference of two random vectors to a third
    Perform crossover between mutant and current individual to create trial vector
    Evaluate fitness of trial vector
    if trial vector is better **then**
      Replace individual with trial vector
    end if
  end for
end for
return best individual found

Differential Evolution (DE) is a stochastic, population-based method useful for global optimization problems. Unlike GA, DE generates new candidate solutions

by adding the scaled difference between two population members to a third one. This allows for more diverse mutations and broad exploration of the parameter space.

In each generation, DE performs mutation and crossover: a new candidate vector is generated and mixed with an existing target vector. If the fitness of the new candidate is better (lower), it replaces the target in the next generation. This greedy selection mechanism helps maintain convergence toward better solutions.

In our case, DE is configured with a population size of 15, a mutation factor range of 0.5 to 1.0, and a recombination rate of 0.7 Sci (2021). These values strike a balance between convergence speed and avoiding premature stagnation. The optimization is handled by the differential_evolution function in SciPy:

```
result = differential_evolution(
        func=fitness,
        bounds=bounds,
        args=(frames, s0, vehicle_class),
        strategy='best1bin',
        maxiter=100,
        popsize=15,
        tol=0.01,
        mutation=(0.5, 1.0),
        recombination=0.7,
        seed=42
    )
```

Differential Evolution is particularly effective for high-dimensional, non-differentiable, or rugged search spaces. It tends to outperform gradient-based methods in finding global minimum but requires more computational time.

6.2 Driver Classification

To assess the effect of lane changes on IDM-based classification, the process was conducted in two variants. In the first variant, all drivers were included, even those performing lane changes, and the entire trajectory was used for calibration. This choice reflects realistic conditions but also means that lane-change segments, which can alter the observed speed and gap, may influence the parameter estimates. In the second variant, only drivers who maintained a constant preceding vehicle throughout the recording were considered, ensuring that parameter fitting was unaffected by lateral maneuvers.

After optimizing the IDM parameters using all three methods, the performance

of each approach was evaluated based on the mean relative error across all drivers. GA consistently produced lower average error values compared to MLE and DE, especially for drivers with higher variability in observed speed and gap. Due to its robustness and higher accuracy, we chose the GA-derived parameter sets as the basis for driver classification.

Once the parameters $\theta = (T, a, b, v_0)$ were estimated for all drivers using GA, we applied two classification strategies to label driver behavior as *aggressive*, *normal*, or *cautious*. Each method uses the same feature space but a different decision mechanism.

6.2.1 Rule-Based Classification

The first method uses a simple rule-based system based on thresholds applied to three parameters: time headway T, acceleration a, and deceleration b. The logic is directly derived from domain knowledge of driving behavior: aggressive drivers tend to follow closely (low T), accelerate quickly (high a), and brake sharply (high b). Cautious drivers prefer larger headways and avoid high acceleration or deceleration.

The classification logic is as follows:

```
if T < 1.0 and a > 2.0 and b > 2.0:
    label = "aggressive"
elif T > 2.5 and a < 1.5 and b < 2.0:
    label = "cautious"
else:
    label = "normal"
```

This rule set is intentionally simple, allowing for interpretability and direct control over class boundaries.

6.2.2 K-Means Clustering

The second method applies an unsupervised learning technique, K-Means clustering—to the normalized parameter vectors $[T, a, b, v_0]$. Normalization ensures that all features contribute equally to the distance calculations during clustering.

K-Means was selected for its computational efficiency, interpretability, and suitability for partitioning continuous-valued data into distinct behavioral categories. The number of clusters is fixed at $k = 3$, aligning with the target classification labels: *aggressive*, *normal*, and *cautious*.

After the clustering process, behavioral labels are assigned based on the properties of the resulting centroids. The classification follows a multi-criteria decision

rule where clusters are first sorted by time headway (T), then by acceleration (a), deceleration (b), and desired velocity (v_0). The cluster with the lowest mean T is interpreted as aggressive, the cluster with the highest mean T as cautious, and the remaining cluster as normal. This decision rule reflects the assumption that more aggressive drivers tend to maintain shorter following times.

```
cluster_stats.sort
(key=lambda x:(-x[1][1], x[1][0], -x[1][2], -x[1][3]))

# labels:
cluster_labels = {
    cluster_stats[0][0]: 'Aggressive',
    cluster_stats[1][0]: 'Normal',
    cluster_stats[2][0]: 'Cautious',
}
```

Each driver's assigned cluster is then mapped to its corresponding behavioral label. Unlike the rule-based method, K-Means can capture more nuanced patterns and interactions between features, although it is not directly interpretable.

By applying both approaches, we can compare interpretable rule-based decisions with data-driven groupings. In further evaluation, the agreement between the methods provides insight into classification robustness and potential mislabelings.

6.3 Visualization

To evaluate the quality of the parameter calibration and to better interpret driver behavior, several visual analyses are performed. These include comparisons between simulated and observed data, as well as plots related to the clustering results.

First, the model-derived acceleration is compared against the observed acceleration ($XAcc$) from the dataset. For each driver, the actual and simulated acceleration curves are plotted over time. A close match between the curves indicates that the driver's behavior was well captured by the model.

Second, position trajectories are reconstructed using both the estimated and the actual acceleration values. The simulated velocity is updated iteratively using the IDM-derived acceleration and integrated at a fixed time step ($dt = \frac{1}{25}$ s) to produce the simulated position. A parallel calculation is performed using the recorded $XAcc$ to generate the ground truth trajectory. The Root Mean Squared Error (RMSE) and percentage error relative to the mean position are computed to

quantify the deviation between simulated and actual trajectories. This comparison is independent of the fitness function and serves as a secondary validation metric for the model.

In addition to accuracy plots, the clustering results are visualized using two complementary approaches. For the K-Means classification, the 4D parameter space (T, a, b, v_0) is reduced to two dimensions (either via PCA or by selecting T and a) to show how driver types are separated in the learned clusters. These visualizations indicate that the clustering algorithm is able to capture meaningful patterns.

Furthermore, a pairplot analysis displays the distributions and pairwise relationships between all estimated IDM parameters. Each data point is colored by its assigned driver type (aggressive, normal, cautious), enabling clear interpretation of how different driving styles correspond to distinct parameter combinations.

Together, these visualizations provide a comprehensive picture of both model performance and the diversity of driver behavior captured by the calibrated IDM parameters.

7 Evaluation

7.1 Overview and Baseline

This chapter evaluates how well different optimization methods calibrate the IDM to real-world driving behavior and whether the estimated parameters are suitable for downstream driver classification.

As baseline references, Kesting and Treiber (2008) demonstrated successful IDM calibration using multiple distance-based objective functions. They reported reproduction errors between 11% and 29%, confirming the robustness of IDM compared to other car-following models. However, their work focused solely on calibration performance and did not use the estimated parameters for behavior classification.

7.2 Data Description

The evaluation uses the highD dataset, which includes 60 highway traffic recordings from drone video. Each recording is stored in a `.pkl` file that contains per-frame information for all vehicles, including speed, acceleration, and relative distances. Each recording also has a corresponding `tracks_meta.csv` file that includes static metadata for each vehicle, such as vehicle type, number of frames, and average speed.

From the `.pkl` file, we extract for each ego vehicle the following variables per time step: longitudinal velocity v_t, gap to the leading vehicle s_t, relative velocity Δv_t, and measured acceleration $Xacc$. These are combined with the metadata using the unique vehicle ID to form a trajectory dataset for each driver:

$$D_{\text{driver}} = \{(v_t, s_t, \Delta v_t, XAcc_t)\}_{t=1}^{T}$$

This dataset is used to estimate IDM parameters $\theta = (T, a, b, v_0)$ for each driver via calibration.

The road segments in the dataset consist of four lanes, two in each driving direction. To ensure consistency, velocities and accelerations of vehicles driving in the opposite direction (with negative values) are inverted as follows:

Listing 7.1: Direction Correction for Longitudinal Values

```
driver_data['xVelocity'] = abs(driver_data['xVelocity'])
```

If no leading vehicle is detected (`precedingId == 0`), we assign a default headway of 250 meters to reflect the maximum visible distance for drivers:

Listing 7.2: Handling Missing Leading Vehicle

```
driver_data.loc[driver_data['precedingId'] == 0, 'dhw'] = 250.0
```

7.3 Calibration Setup

To determine which calibration strategy is most suitable for behavior classification, we systematically test three optimization algorithms: Genetic Algorithm (GA), Maximum Likelihood Estimation (MLE), and Differential Evolution (DE). Each algorithm is applied using three objective functions: Relative error (F_{rel}), Absolute error (F_{abs}), Mixed error (F_{mixed}).

All nine combinations are applied to 90 drivers from the first highD recording. The resulting parameters are stored as both `.txt` and `.csv` files for subsequent behavior classification.

Below is an example of the processed output for one time step of a driver trajectory:

Listing 7.3: Example frame data output

```
# .txt output
 Frame Data:
 Frame: 1471, (PID:75 => v:25.47)
 v: 34.41, delta_v: 8.94, s: 68.79, s*_GA: 49.97
 [IDM_GA: 0.0362 | IDM_MLE: 0.0362 |
 IDM_DE: 0.0362 | XAcc: -0.3900 m/s^2]

 # .csv output
rec_id,driver_id,GA_T,GA_a,GA_b,GA_v0,MLE_T,MLE_a,MLE_b,
MLE_v0,DE_T,DE_a,DE_b,DE_v0
1,11,0.5,0.1,5.0,41.667,0.5,0.1,5.0,41.667,0.5,0.1,5.0,41.667
1,27,0.5,0.1,5.0,37.066,0.5,0.1,5.0,37.065,0.5,0.1,5.0,37.062
```

7.4 Evaluation Metrics

Calibration quality is assessed using three complementary evaluation metrics, illustrated in Figures 7.1, 7.2, and 7.3.

Gap error measures the deviation between the simulated and observed headway s_t. Accurate minimization of gap error typically improves both velocity and acceleration predictions, as shown by Kesting and Treiber (2008). As the primary calibration target, it directly reflects car-following behavior and serves as the basis of the objective function F_{rel} used in the optimization process. (Figure 7.1).

Acceleration error measures how well the simulated acceleration matches the observed longitudinal acceleration $XAcc_t$. It reflects the local accuracy of the model in replicating driver control (Figure 7.3).

Position error is computed by integrating acceleration over time and comparing the resulting simulated trajectory to the actual trajectory. It captures the cumulative deviation in motion dynamics (Figure 7.2).

These metrics provide different views on model fidelity: acceleration error focuses on short-term control, position error on trajectory realism, and gap error on interaction fidelity. Following Kesting and Treiber (2008), we prioritize minimizing gap error, as doing so typically reduces both velocity and acceleration errors due to the coupled nature of IDM.

By comparing all combinations of algorithms and objective functions using these three metrics, we can identify which configuration yields the most accurate and stable parameter estimation. This is essential for reliable downstream classification, as errors in calibration can affect the clustering of drivers by behavior.

The full results are reported in Chapter Results.

7.5 Best Performing Calibration Strategy

Based on the experimental results presented in Chapter Results, we identify Genetic Algorithm combined with the relative error objective F_{rel} as the most effective calibration strategy. This combination achieves the lowest average gap error across drivers while maintaining competitive performance in acceleration and position accuracy.

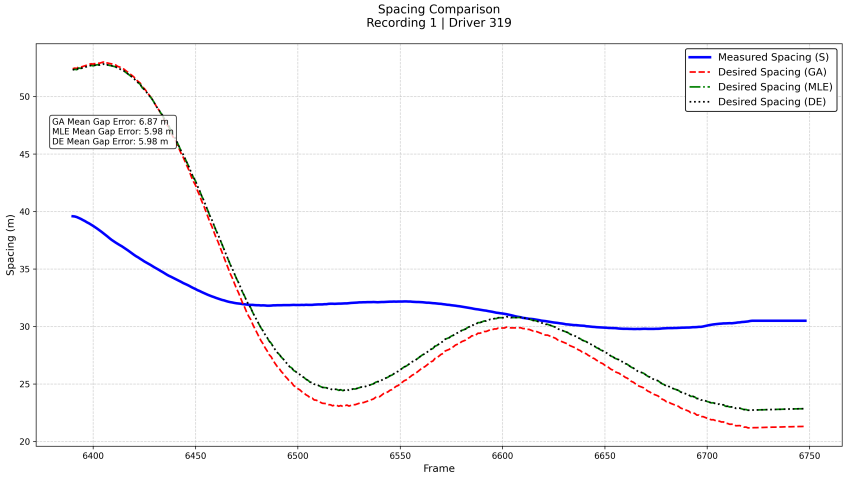

Figure 7.1: Gap error across calibration strategies

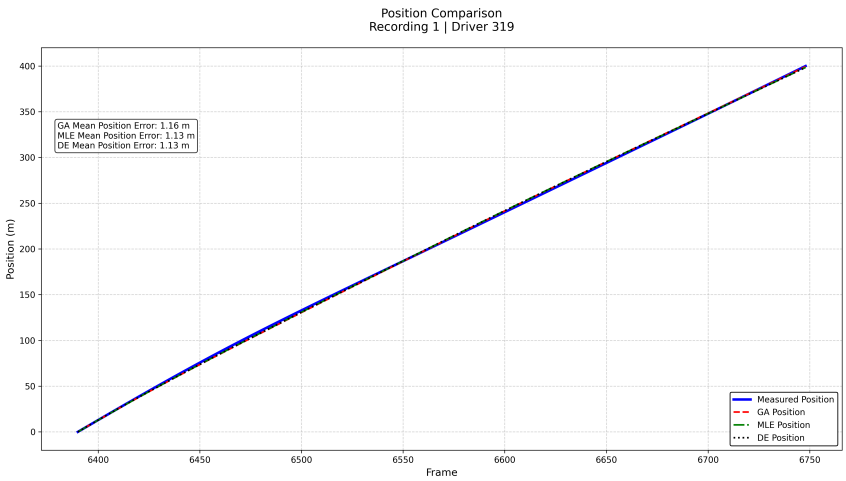

Figure 7.2: Position error across calibration strategies

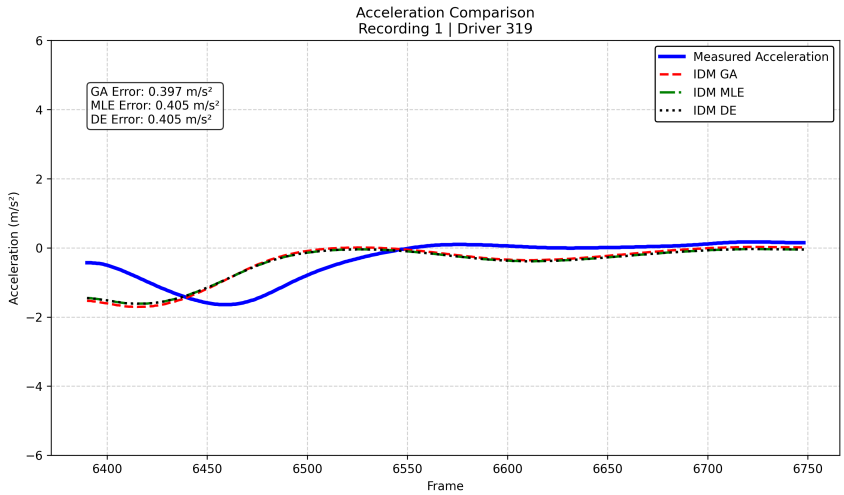

Figure 7.3: Acceleration error across calibration strategies

This selected configuration Genetic Algorithm + F_{rel} is used in all subsequent experiments for behavior classification.

7.6 Results

7.6.1 Calibration Results

To assess the performance of each optimization algorithm under different calibration objectives, we evaluate them across three independent metrics: acceleration error, position error, and gap error. Each objective function (relative error (F_{rel}), absolute error (F_{abs}), and mixed error (F_{mixed})) was used to calibrate parameters for 90 drivers using three algorithms: Genetic Algorithm (GA), Maximum Likelihood Estimation (MLE), and Differential Evolution (DE).

Runtime Comparison

Table 7.1: Runtime per driver (in seconds) for each algorithm and objective function

Objective Function	GA	MLE	DE
F_{rel}	38.0486	0.7068	5.3803
F_{abs}	41.1085	1.4806	2.4436
F_{mixed}	37.2756	1.2265	3.0789

Results for F_{rel} Objective

Table 7.2: Evaluation results for F_{rel} objective function. The gap error row for the Genetic Algorithm (GA) is highlighted, as gap (s_t) is the primary calibration target.

Metric	Algorithm	Mean	Median	Min	Max	Std. Dev.
Acceleration (m/s^2)	GA	1.094	0.400	0.023	11.838	1.741
	MLE	1.086	0.401	0.017	11.838	1.734
	DE	1.119	0.377	0.017	11.838	1.757
Position (%)	GA	5.69	2.27	0.11	47.25	8.01
	MLE	5.56	2.38	0.09	47.25	7.97
	DE	5.60	2.27	0.09	47.25	8.08
Gap (m)	GA	71.765	32.752	1.377	620.336	104.608
	MLE	72.050	32.407	1.540	620.574	106.400
	DE	74.702	35.547	1.541	620.571	106.797

Results for F_{abs} Objective

Table 7.3: Evaluation results for F_{abs} objective function

Metric	Algorithm	Mean	Median	Min	Max	Std. Dev.
Acceleration (m/s^2)	GA	1.304	0.403	0.018	11.838	2.060
	MLE	1.395	0.418	0.018	11.838	2.110
	DE	1.383	0.418	0.018	11.838	2.093
Position (%)	GA	7.59	2.18	0.10	67.08	13.10
	MLE	7.25	2.41	0.10	67.01	12.92
	DE	7.31	2.38	0.10	67.06	12.94
Gap (m)	GA	70.280	32.574	1.605	619.426	92.046
	MLE	69.945	32.217	1.562	617.999	92.164
	DE	69.845	32.061	1.562	619.199	93.302

Results for F_{mixed} Objective

Table 7.4: Evaluation results for F_{mixed} objective function

Metric	Algorithm	Mean	Median	Min	Max	Std. Dev.
Acceleration (m/s^2)	GA	1.230	0.400	0.024	11.838	1.827
	MLE	1.260	0.398	0.018	11.838	1.838
	DE	1.232	0.410	0.018	11.838	1.829
Position (%)	GA	6.52	2.33	0.10	47.25	9.85
	MLE	6.16	2.40	0.09	47.25	9.64
	DE	6.43	2.38	0.09	47.25	9.84
Gap (m)	GA	73.125	33.171	1.535	632.936	107.910
	MLE	74.357	32.264	1.542	633.384	108.827
	DE	75.005	32.679	1.590	633.170	108.565

7.6.2 Classification Results

Rule-based and unsupervised classification methods were applied to analyze driver behavior using calibrated IDM parameters. The analysis was conducted in two stages:

- All drivers (1067 total) including those who performed lane changes.
- A subset of drivers who did not change lanes during their trajectories.

Drivers with Lane Changes

Rule-Based Classification Threshold-based rules, as described in Chapter Methods, were applied to all 1067 drivers.

Table 7.5: Driver class distribution based on rule-based classification (lane-changing)

Driving Style	Number of Drivers
Aggressive	222
Normal	752
Cautious	93
Total	1067

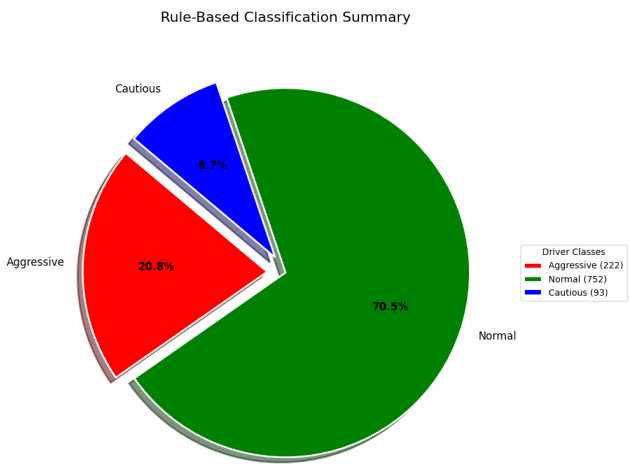

Figure 7.4: Driver class distribution by rule-based classification (lane-changing)

K-Means Clustering K-means clustering with $k = 3$ was applied to the same set of 1067 drivers.

Table 7.6: Cluster mean parameter values from K-means (lane-changing)

Cluster (Label)	**T**	**a**	**b**	v_0 **(m/s)**
Aggressive (0)	1.020	4.600	3.875	40.134
Cautious (1)	1.720	0.759	4.401	36.372
Normal (2)	3.345	1.737	0.887	38.215

Table 7.7: Driver distribution across K-means clusters (lane-changing)

Driving Style (Cluster)	**Number of Drivers**
Aggressive (Cluster 0)	359
Cautious (Cluster 1)	424
Normal (Cluster 2)	284
Total	1067

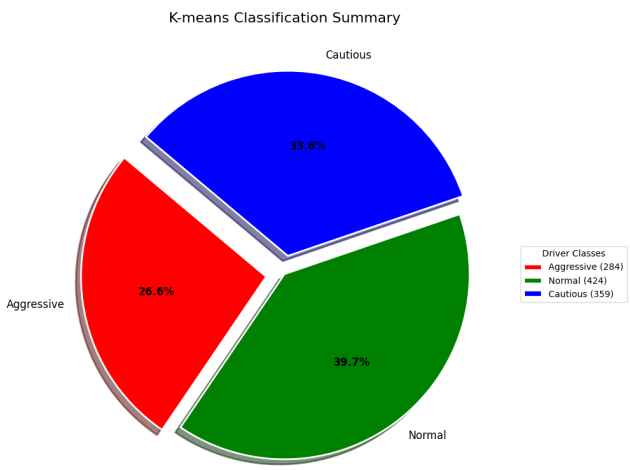

Figure 7.5: Driver class distribution by K-means clustering (lane-changing)

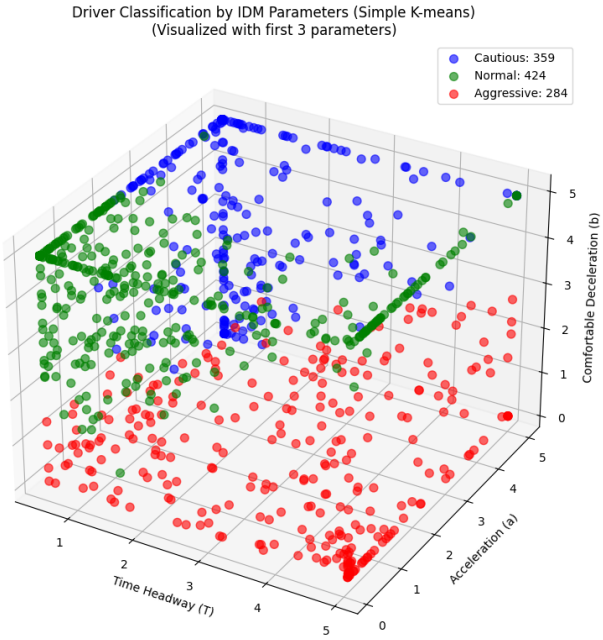

Figure 7.6: 3D visualization of K-means clusters in parameter space (lane-changing)

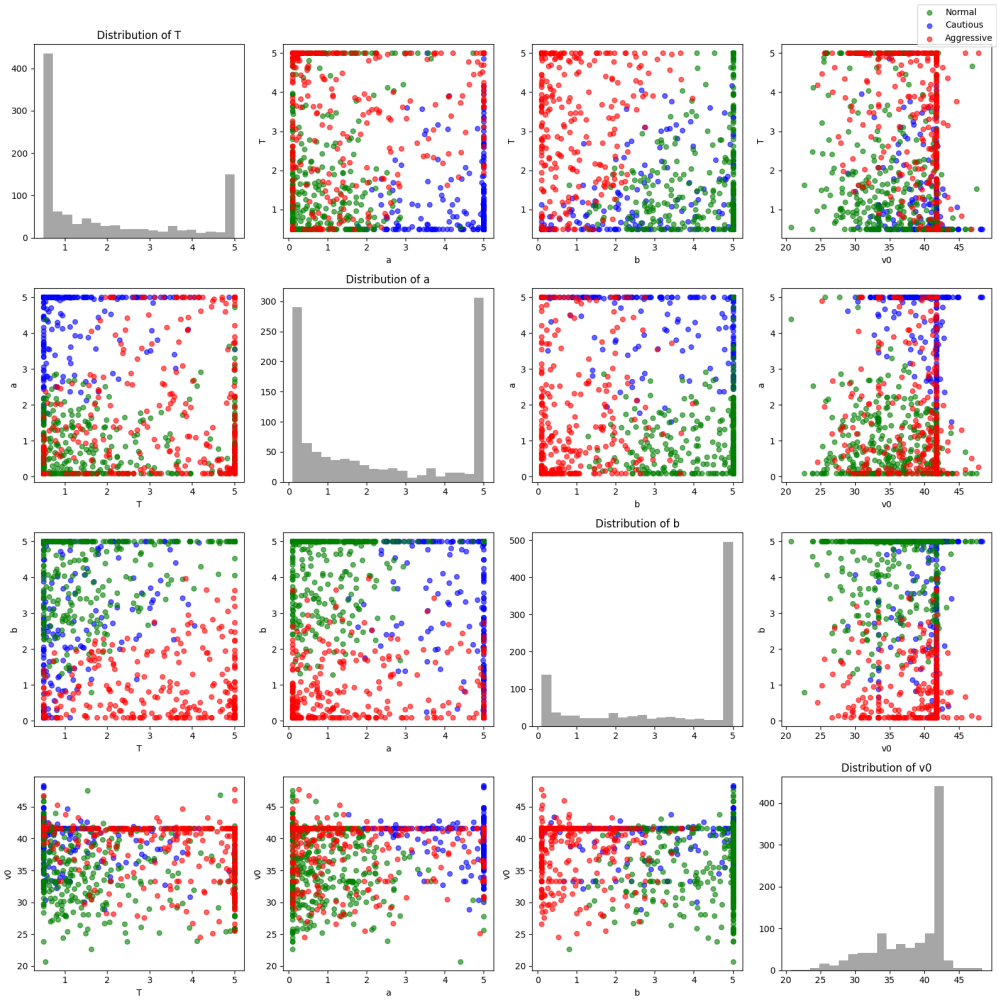

Figure 7.7: Pairwise distributions of IDM parameters across K-means clusters (lane-changing)

Drivers without Lane Changes

Rule-Based Classification The rule-based classification method was also applied to a subset of 464 drivers who did not change lanes during their trajectories.

Table 7.8: Driver class distribution based on rule-based classification (no lane changes)

Driving Style	Number of Drivers
Aggressive	28
Normal	392
Cautious	44
Total	464

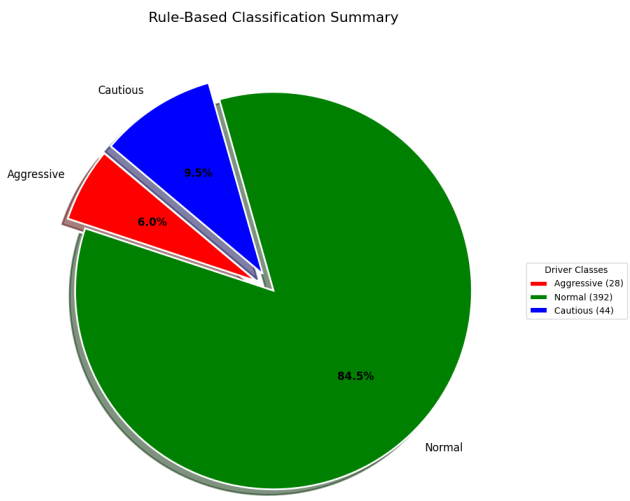

Figure 7.8: Driver class distribution by rule-based classification (no lane changes)

K-Means Clustering K-means clustering with $k = 3$ was applied to the same no-lane-change subset.

Table 7.9: Cluster mean parameter values from K-means (no lane changes)

Cluster (Label)	**T**	**a**	**b**	v_0 **(m/s)**
Cautious (Cluster 0)	1.176	0.736	3.771	36.109
Normal (Cluster 1)	3.914	1.090	1.658	35.794
Aggressive (Cluster 2)	2.206	4.282	2.514	39.470

Table 7.10: Driver distribution across K-means clusters (no lane changes)

Driving Style (Cluster)	**Number of Drivers**
Cautious (Cluster 0)	240
Normal (Cluster 1)	112
Aggressive (Cluster 2)	112
Total	464

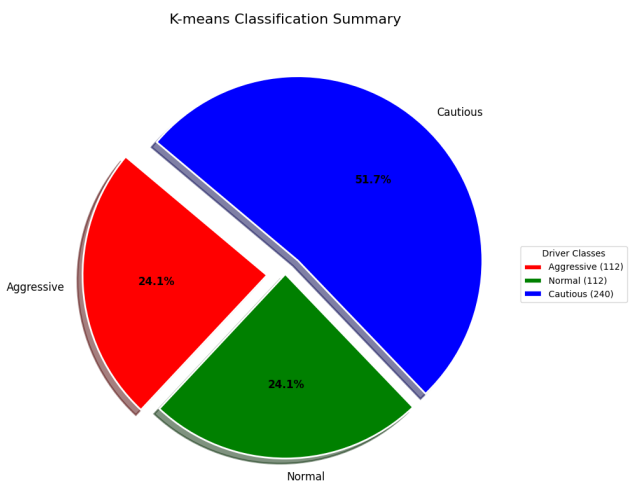

Figure 7.9: Driver class distribution by K-means clustering (no lane changes)

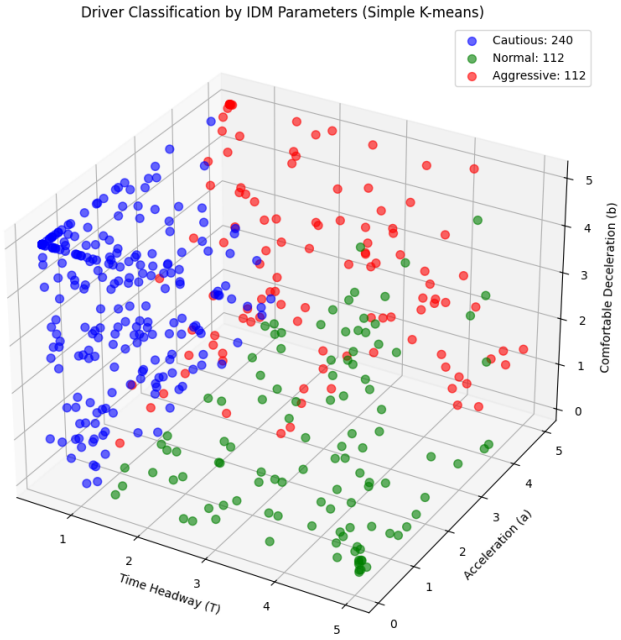

Figure 7.10: 3D visualization of K-means clusters in parameter space (no lane changes)

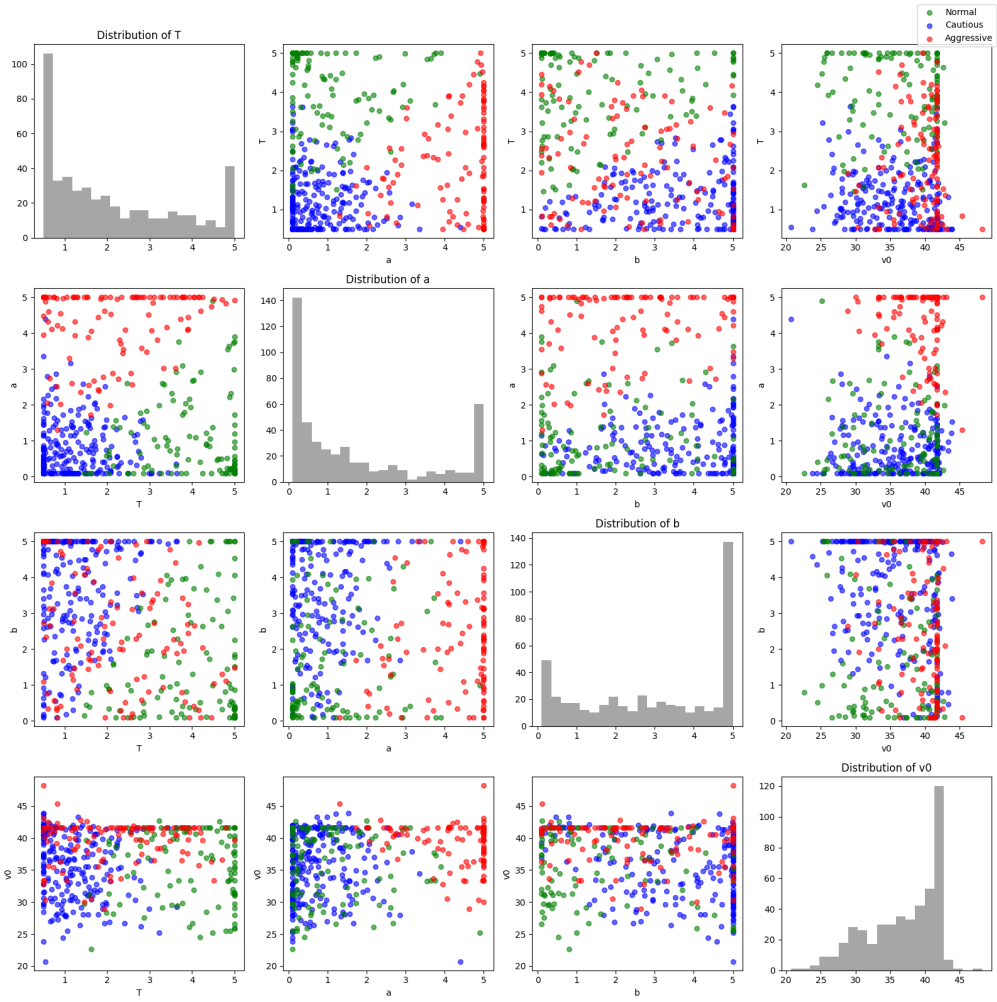

Figure 7.11: Pairwise distributions of IDM parameters across K-means clusters (no lane changes)

7.7 Discussion

7.7.1 Interpretation of Results

The calibration results demonstrate that all three algorithms GA, MLE, and DE perform comparably across the evaluated objective functions. The acceleration and position errors remain relatively low for all methods, particularly in their median values. This indicates that the estimated IDM parameters lead to trajectories closely aligned with real driver behavior under typical conditions.

Among the algorithms, the Genetic Algorithm (GA) achieved the best performance in terms of minimizing the gap error $s(t)$, with a mean value of 71.77 m, as shown in Table 7.2. This result highlights the GA's capacity to search the parameter space more effectively in the context of noisy headway data, making it a strong candidate for calibration focused on safety-relevant measures.

Gap errors, in general, exhibit higher variance than other metrics. This outcome is expected due to the intrinsic sensitivity of the gap-based calibration approach to small deviations in vehicle trajectories. Nevertheless, minimizing the gap error remains a valid and effective target. As Kesting and Treiber (2008) write, reducing error in the headway $s(t)$ also leads to lower velocity and acceleration errors.

Moreover, when comparing different error functions, the relative error objective (F_{rel}) offers slightly more stable and consistent results across all three algorithms. The absolute and mixed error metrics also yield competitive calibration quality, suggesting that the choice of error function should consider both the intended use case and data quality.

7.7.2 Classification Analysis

The classification analysis provides consistent identification of three driver types (Aggressive, Normal and Cautious) through both rule-based and K-means clustering approaches. Figure 7.12 shows the distribution patterns from rule-based classification, while Figure 7.13, Figure 7.14 and Figure 7.15 present the complete parameter comparison across driving styles.

The rule-based classification in Figure 7.12 reveals significant behavioral differences between scenarios. Lane-changing situations contain nearly three times as many Aggressive drivers (20.8%) compared to non-lane-changing scenarios (6.0%). Normal driving dominates both cases but shows greater prevalence in non-lane-changing situations (84.5% vs. 70.5%). Cautious drivers maintain consistent minority representation across scenarios, comprising 8.7% of lane-changing and 9.5% of non-lane-changing cases.

For lane-changing drivers, the Aggressive cluster in Figure 7.13 demonstrates characteristically low time headways (T = 1.020 s) coupled with high acceleration capability (a = 4.600 m/s²), strong deceleration (b = 3.875 m/s²), and elevated desired speeds (v0 = 40.134 m/s). In contrast, the Cautious in Figure 7.15 group exhibits higher time headways (T = 1.720 s) with more moderate acceleration (a = 0.759 m/s²), stronger deceleration tendencies (b = 4.401 m/s²), and lower preferred speeds (v0 = 36.372 m/s). Normal drivers in Figure 7.14 occupy an intermediate position with balanced parameters (T = 3.345 s, a = 1.737 m/s², b = 0.887 m/s², v0 = 38.215 m/s).

The non-lane-changing scenarios show distinct behavioral patterns. Aggressive

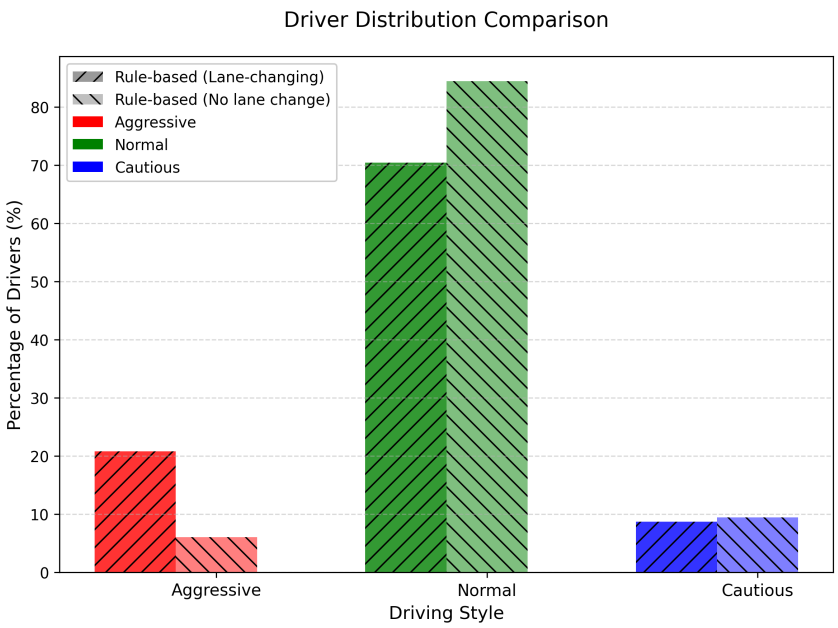

Figure 7.12: Distribution of driving styles using rule-based classification, comparing lane-changing and non-lane-changing scenarios.

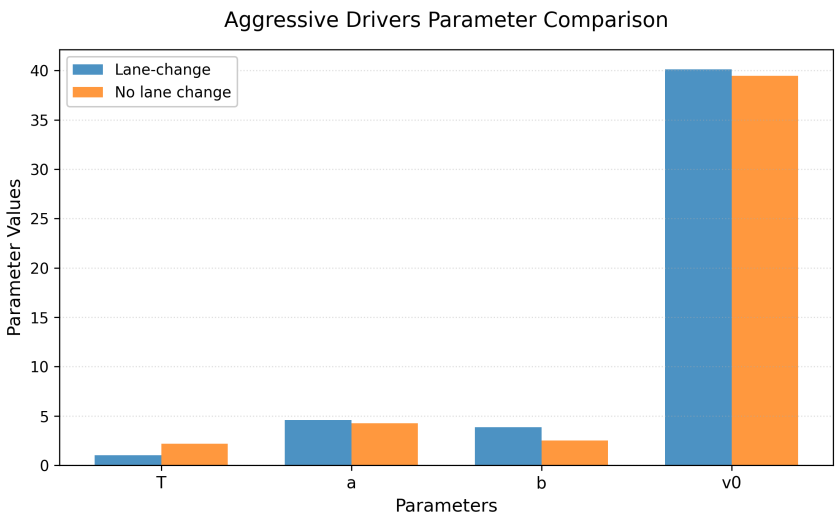

Figure 7.13: K-means parameters for Aggressive drivers.

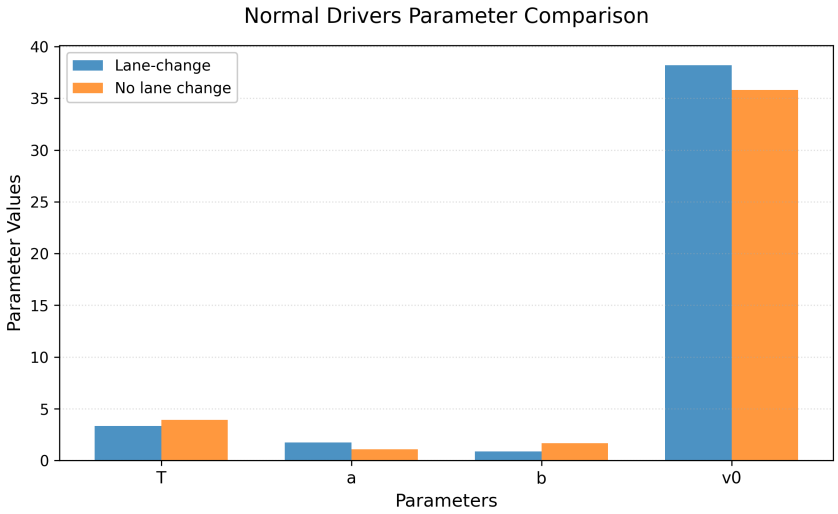

Figure 7.14: K-means parameters for Normal drivers.

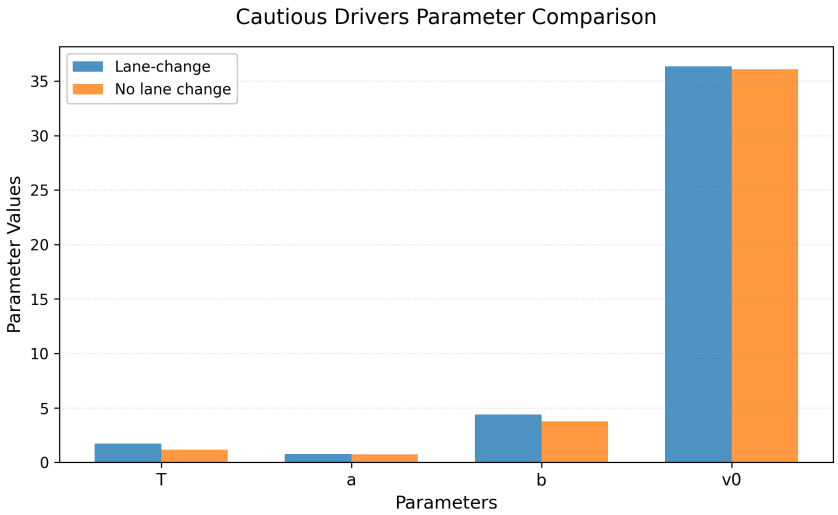

Figure 7.15: K-means parameters for Cautious drivers.

drivers maintain their high acceleration characteristics (a = 4.282 m/s²) but with notably longer time gaps (T = 2.206 s) compared to their lane-changing counterparts. Normal drivers in this scenario display the most balanced parameter set (T = 3.914 s, a = 1.090 m/s², b = 1.658 m/s², v0 = 35.794 m/s), while Cautious drivers adopt extremely conservative behavior with the lowest acceleration values (a = 0.736 m/s²) and moderate time headways (T = 1.176 s).

These differences underline the influence of lane changes on driving style classification. Non-lane-changing trajectories are generally more stable, resulting in tighter and more balanced clusters. This suggests that separating lane-changing and non-lane-changing behavior during calibration and classification improves interpretability and robustness. The visualization in Figures 7.13 and 7.15 supports these findings, clearly showing the spatial separation of clusters in parameter space.

These findings indicate that lane-changing behavior is closely associated with driver aggressiveness. As shown in Figure 7.13, drivers who change lanes often are more likely to be classified as Aggressive, which matches the higher numbers and stronger parameter values seen in the lane-changing group. It cannot be determined for sure what causes what, but both factors likely play a role: aggressive drivers tend to change lanes more and frequent lane changes might make driving even more aggressive.In reality, aggressive drivers also perform more overtakes, so this connection is consistent with typical driving behavior.

7.7.3 Limitations

Several limitations were encountered during the analysis. IDM assumes a constant leading vehicle and does not explicitly model lane changes. Because of that, it often fails to represent real-world situations with rapid decelerations or merging traffic. In some cases from the dataset, this limitation led to large calibration errors reaching over 55% maximum error in certain segments, especially when the vehicle performed unexpected maneuvers that IDM could not describe well.

Another key issue appears in the clustering results. As shown in the pairplot visualization (Figure 7.7), a large number of drivers reach the imposed upper bounds of parameters such as acceleration a and desired speed v_0. This saturation suggests that the model tries to compensate for its limited structure by pushing parameters to their limits. It reduces the ability to distinguish between truly different driving behaviors and those merely shaped by model constraints. This is further complicated by the presence of short, noisy, or incomplete trajectories, which distort both calibration and classification outcomes.

7.7.4 Future Work

Future improvements should consider using more flexible car-following models that can better handle dynamic traffic interactions, such as lane changes, varying lead vehicle behavior, or stop-and-go traffic. Adding parameters to the model or switching to an alternative like the Adaptive IDM (AIDM) may improve fit. For example, the adaptive model proposed in Alhariqi et al. (2022) adjusts two scaling parameters (λ_a and λ_T) and achieves a significant reduction in error while improving driving comfort and realism. They reported that error values dropped from 2.11 to 0.95 in one experiment, showing the potential of minimal extensions to IDM.

8 Conclusions

This thesis explored the calibration and interpretation of driver behavior using the Intelligent Driver Model (IDM). The work focused on two key aspects accurate parameter estimation and classification of driving styles.

The calibration results showed that all three tested optimization methods Genetic Algorithm (GA), Maximum Likelihood Estimation (MLE), and Differential Evolution (DE) achieved comparable accuracy. Across absolute, relative, and mixed objective functions, the estimated parameters produced trajectories with low error in acceleration and position. GA performed best in minimizing the headway gap error $s(t)$, confirming its strength in handling noisy or uncertain data, especially where safety-relevant behavior is prioritized.

The relative error objective function delivered slightly more stable calibration results across methods. Notably, minimizing headway gap error remained effective, even when its values exhibited higher variance. This finding aligns with Kesting and Treiber (2008), which emphasized that reducing error in $s(t)$ also lowers velocity and acceleration deviations.

For classification, the analysis used both rule-based thresholds and K-means clustering on calibrated IDM parameters. Results consistently revealed three main driving styles: Aggressive, Normal, and Cautious. These patterns were supported across drivers who changed lanes and those who did not. Splitting the data by lane changes proved helpful, as drivers without lane changes showed tighter, more interpretable clusters with lower parameter variance

The classification also highlighted a challenge: many drivers reached upper bounds in parameters such as acceleration and desired speed. This suggests that the IDM, while simple and interpretable, may be too rigid to capture all variations in real-world behavior. The model sometimes pushed parameters to unrealistic limits in an attempt to compensate for dynamics it cannot describe sudden braking, merging, or short noisy trajectories.

In summary, the IDM with basic calibration and classification methods can describe and group typical driving behavior reasonably well. Still, it shows clear limits under more complex or dynamic situations.

Future work should explore enhanced models such as the Adaptive IDM Alhariqi et al. (2022), which introduces minimal extensions to better match real data while preserving interpretability. Additionally, including more advanced features,

such as lane-change modeling, could improve both calibration accuracy and driver classification robustness.

Bibliography

M. Treiber, A. Hennecke, and D. Helbing, "Congested traffic states in empirical observations and microscopic simulations," *Physical review E*, vol. 62, no. 2, p. 1805, 2000.

M. Treiber and A. Kesting, *Traffic flow dynamics*. Springer, 2012, vol. 1.

S. Bouhsissin, N. Sael, and F. Benabbou, "Driver behavior classification: A systematic literature review," *IEEE Access*, vol. 11, pp. 14 128–14 153, 2023.

A. Kesting and M. Treiber, "Calibrating car-following models by using trajectory data: Methodological study," *Transportation Research Record*, vol. 2088, no. 1, pp. 148–156, 2008.

M. Treiber and A. Kesting, "Microscopic calibration and validation of car-following models–a systematic approach," *Procedia-Social and Behavioral Sciences*, vol. 80, pp. 922–939, 2013.

A. Alhariqi, Z. Gu, and M. Saberi, "Calibration of the intelligent driver model (idm) with adaptive parameters for mixed autonomy traffic using experimental trajectory data," *Transportmetrica B: transport dynamics*, vol. 10, no. 1, pp. 421–440, 2022.

X. Wen, Z. Cui, and S. Jian, "Characterizing car-following behaviors of human drivers when following automated vehicles using the real-world dataset," *Accident Analysis & Prevention*, vol. 172, p. 106689, 2022.

B. Stoler, I. Navarro, J. Francis, and J. Oh, "Seal: Towards safe autonomous driving via skill-enabled adversary learning for closed-loop scenario generation," *arXiv preprint arXiv:2409.10320*, 2024.

J. H. Holland, "Genetic algorithms," *Scientific American*, Jul. 1992.

R. Ali, "Introduction to genetic algorithms: Python, example, code, optimizing success through evolutionary computing," *Medium*, May 2023, accessed: 2025-05-12. [Online]. Available: https://medium.com/@Data_Aficionado_1083/genetic-algorithms-optimizing-success-through-evolutionary-computing-f4e7d452084f

D. Madej, "How to implement a maximum likelihood estimation code for any distribution," *Python in Plain English*, Nov 2021, accessed: 2025-05-12. [Online]. Available: https://python.plainenglish.io/scipy-has-failed-you-do-maximum-likelihood-estimation-by-yourself-34145d71b63f

SciPy v1.7.0 Documentation: differential_evolution, SciPy, 2021. [Online]. Available: https://docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.differential_evolution.html